The Runbook, in Plain English

What this app really is, why it needs two programs, and how to run it without breaking it.

1. What is this thing, actually?

Strip away the jargon and it's an automated **research intern**. You point it at the internet and it comes back with a binder of notes and a shortlist.

Concretely, it does seven things in a row:

1. Takes a list of websites — either you supply them, or it *searches* for them.
2. Visits those pages automatically, at a polite speed.
3. Reads them and works out which **companies** they belong to.
4. Spots the repetitive human tasks those companies clearly do — those are called **workflows**.
5. **Scores** each workflow from 0–100 across seven dimensions.
6. Turns the strongest ones into **opportunities** — "an AI agent could plausibly do this job here."
7. Shortlists the very best as **qualified leads**, for a human to review.

> **The analogy that makes it click**
>
> You hire an intern and say: "Go look at 200 plumbing company websites in Leeds. Come back and tell me which ones waste the most time on manual paperwork, because that's what we sell automation for." That intern is this app. The "binder of notes" is the `/research` page. The "shortlist" is `/qualified-leads`.

2. The one idea that explains everything: there are TWO programs

This is the single biggest source of confusion, and it's not obvious from the outside.

`npm run dev` — the reception desk

This is the website you see at localhost:3000. It takes your requests and writes them down as "jobs".

It does not do any of the actual work.

`npm run worker` — the engine room

This is a separate background process. It reads those jobs and actually goes out and fetches the web pages, parses them, and writes results.

> **If you only start the website, nothing happens. Ever.**
>
> Jobs will pile up in the queue and sit there forever, and it will look like the app is broken. It isn't — you just never turned the engine on. This is the number-one beginner mistake with this codebase.

> **And run only ONE worker.**
> In development everything is saved into a single file, `.data/scraper.json`. It's a plain JSON file with no locking — like one physical notebook. If two workers write at the same time they overwrite each other and the data corrupts. The app is designed to detect a running worker and refuse to start a second one, so if it refuses, that's a feature, not a bug.

3. First-time setup (once per machine)

```
npm install --legacy-peer-deps
cp .env.example .env.local
npm run db:generate
```

Command	What it actually means
`npm install --legacy-peer-deps`	Downloads all the third-party code the app needs. **The flag is not optional.** One testing tool and the pinned TypeScript definitions disagree about versions; without the flag, npm stops and refuses to install anything.
`cp .env.example .env.local`	Copies the template settings file into your own private settings file. `.env.local` is the one the app actually reads, and it's deliberately excluded from git so your secrets never get committed.
`npm run db:generate`	Builds the database client code. You need this **even if you're not using a database**, because parts of the code import it at startup and will crash without it.

Then fill in four settings in `.env.local`

```
EJENTIC_ADMIN_EMAIL=admin@ejentic.local
EJENTIC_ADMIN_PASSWORD=change-me
SESSION_SECRET=<long random string>
BRAVE_SEARCH_API_KEY=          # optional, but recommended
```

Setting	Plain meaning
Admin email + password	Your login. That's it.
`SESSION_SECRET`	A secret used to sign the "you are logged in" cookie. **Generate it, don't invent it** — a guessable secret means a forgeable cookie. The command in the original runbook prints a properly random one.
`BRAVE_SEARCH_API_KEY`	Makes the "find me websites" search feature work properly. Without it that feature falls back to weaker providers and may just fail.

> **The non-obvious bit worth remembering: it fails OPEN, not closed**
>
> If you set **none** of the admin variables and have no `DATABASE_URL`, then auth simply reports `DISABLED` and **the dashboard is wide open — no login at all**.
>
> Why would anyone build that? Because the alternative is worse for a developer: one missing config file and you're permanently locked out of your own app with no way in. So the authors chose "annoyingly open" over "uselessly locked". **Fine on your laptop. Never, ever deploy it that way to a real server.**

4. Starting it (every session)

```
npm run dev        # Terminal 1  ->  http://localhost:3000
npm run worker     # Terminal 2  ->  nothing crawls without this
```

Then check `http://localhost:3000/api/health`. It's a plain status readout telling you three things: whether login is on, which search provider is currently active, and which storage backend is in use. If something is misconfigured, this is the fastest place to see it.

> **Restaurant analogy**
>
> The website is the waiter with a notepad. The worker is the kitchen. Opening the restaurant without the kitchen means lots of orders taken and zero food served.

5. Opening it in your browser

This sounds too obvious to need a section — and it's the single thing people actually get stuck on. So here it is, spelled out.

Once the dev server is running, open any browser, click the address bar, and type:

```
localhost:3000
```

Log in, and you're in. That's the whole process. **Bookmark it** (Ctrl+D) and from then on it's one click, forever.

Thing	Does it ever change?
The address `localhost:3000`	**Never.** It's permanent. Type it once, bookmark it, and you never type it again.
The server `npm run dev`	**Yes — it's only alive while it's running.** Close the terminal, restart the machine, or crash it, and the address leads nowhere.

> **"This site can't be reached" almost always means exactly one thing**
>
> The server isn't running. Nothing is broken, and there's nothing wrong with the address — you just haven't turned it on. Go to the project folder and run `npm run dev` again.
>
> Same story after a reboot, after closing the terminal by accident, or the next morning. It's always the same fix.

From a phone or another computer

On the same Wi-Fi, use the **Network** address that the dev server prints when it starts — it looks like:

```
http://172.20.10.2:3000
```

Your number will be different. It's shown in the terminal right beside the Local address every time the server boots, so just copy it from there rather than guessing.

> **It is not on the internet**
>
> There is no domain name to visit — no `ejenticai.com`, nothing you can send a client to. This is a program running on **your** computer, and `localhost` is simply the name your machine uses to talk to itself. Nobody outside your computer can reach it. That's the default, and while you're developing, it's exactly what you want.

The one-second "is it actually up?" check

Open `localhost:3000/api/health`. A wall of JSON means it's alive and tells you the auth, search, and storage status. "Can't be reached" means it's off. That's a faster answer than staring at a job queue wondering whether anything is happening.

6. Creating your first job

Go to `/scraper/jobs/new`.

1. **1. Pick a target type.** For your first run, choose **URL list** with a public URL you control. It's deterministic and fast — you know exactly what should happen, so if something's wrong you can tell. Once that works, try **Web search**.
2. **2. For web search:** fill in name, niche, category, location → press **Find websites** → tick the real results you want. You never type URLs by hand.
3. **3. Set the URL limit.** This is a hard ceiling on how many pages it will ever fetch. Think of it as a spending cap — it cannot be exceeded.
4. **4. Set politeness:** how many workers, requests per second (keep it **1–5**), and how many retries. This is you being a good citizen of the internet — hammering someone's server is how you get your IP blocked.
5. **5. Create.**

> **A design promise worth trusting**
>
> If every search provider fails, **no job is created**, and the app tells you why. It will not silently invent plausible-looking fake results. In a tool whose whole job is producing research, that refusal is a feature — bad data that looks good is far more dangerous than an honest failure.

7. Watching it run

Page	What you see
`/scraper/queue`	The to-do list — URLs still waiting to be fetched.
`/scraper/logs`	A live activity feed. Where you look when you want to know "is it actually doing anything?"
`/scraper/jobs/[id]`	One job's progress, with pause / resume / cancel controls.

8. Reading the results — work through these in order

These six pages are a **funnel**: each one narrows what the previous one produced. Reading them out of order will confuse you.

#	Page	What it is
1	`/campaigns`	A folder that groups jobs under one question — a niche, a region, a sector. ("Plumbers in Leeds.")
2	`/companies`	The companies it discovered. Who's out there.
3	`/research`	The dossier: which pages were read, and the evidence — **with the source URL behind every single quote**. You can always go check. That's the anti-hallucination design.
4	`/workflows`	The repetitive tasks it detected, scored 0–100 across seven dimensions. Press **Evaluate** to score or re-score them.
5	`/opportunities`	Where an AI agent could plausibly take over a workflow.
6	`/qualified-leads`	The shortlist: only score **≥ 75** *and* confidence **≥ 65%**.

> **Qualified leads are a triage aid, not a sales decision**
> The scores get you from "2,000 companies" down to "40 worth a look". They do **not** make the decision for you. **A human reviews before anyone is contacted.** The two thresholds both have to be met — high score with low confidence doesn't make the cut, which is deliberate.

Separately: messages submitted through `/contact` land in `/enquiries`, with new / read / archived states.

9. Before you call it done

```
npm run typecheck
npm run lint      # expect 0 errors, 0 warnings
npm test          # 171 tests
npm run build
```

This isn't optional housework: the project's CI runs **exactly these four** on every push and pull request to `main`. If it passes locally, CI passes. If you skip it, CI is where you find out.

10. Shutting down — order matters

> **Stop the WORKER first (Ctrl+C), then the dev server.**
> Why: the worker is the process that's writing to `.data/scraper.json`. If you kill the server first while the worker is mid-write, you can leave a half-written file behind and corrupt your dev data. Two seconds of discipline saves a confusing debugging session.

11. Safety rules

- **It refuses to crawl localhost, private IP addresses, or cloud metadata endpoints.** This is deliberate protection against a well-known attack (SSRF) where you trick a server into fetching its own internal secrets — e.g. the cloud credentials endpoint at `169.254.169.254`. The worker refuses, and a dedicated test suite enforces it so nobody "helpfully" removes the check later.
- **Use a public URL you own**, or a known public HTML page. Don't point it at a site that hasn't agreed to be crawled.
- **Keep requests/sec low.** You are a guest on someone else's server.
- **Nuclear reset:** delete `.data/scraper.json`. It's the dev store and it's gitignored, so deleting it is safe and reversible — you just lose your local test data.

12. Production shape (later, not now)

```
docker compose up -d      # starts Postgres 17 + Redis 8
# then set DATABASE_URL and REDIS_URL in .env.local
npx prisma migrate deploy
npm run db:seed           # REQUIRED - creates the admin user
npm run dev && npm run worker
```

> **`db:seed` is mandatory here, and the reason is subtle**
>
> Once `DATABASE_URL` is set, **auth is always on** — there's no fail-open any more. So if you forget to seed, you have a running app with a login screen and zero users who can log in. You've locked yourself out of your own deployment. Seed first, always.

> **`BASE_PATH` and `ALLOW_EMBEDDING` are build-time, not run-time**
>
> They must be set in `.env.local` **before** you run `npm run build`. Setting them in your shell at the moment you start the server does nothing. This trips people up constantly: the framework bakes these values into the compiled bundle at build time, so changing them later requires a rebuild.

13. The whole thing in ten seconds

Situation	What to remember
"This site can't be reached"	The server isn't running. `npm run dev`. The address is fine — nothing is broken.
Nothing is happening	You didn't start the worker. Start it.
Worker won't start	One is already running. That's on purpose — find it and stop it.
No login screen	You set no admin variables, so it opened itself. Fine locally, never in production.
Install fails	You forgot `--legacy-peer-deps`.
Search finds nothing	No Brave API key. Optional, but it's what makes search good.
Data looks corrupt	You killed the server before the worker. Delete `.data/scraper.json` and restart.
Locked out in production	You skipped `db:seed` with `DATABASE_URL` set.
Config change ignored	It's `BASE_PATH` or `ALLOW_EMBEDDING` — rebuild.
Results look confident but wrong	Click through to the source URL. Every quote has one. Verify before you act.
Someone's about to be emailed	Stop. The shortlist is a triage aid. A human reviews first.